# AI Developer Agent Modes

Complete guide to different operational modes for AI coding agents and when to use them.

## Agent Modes Overview

### 1. ReAct (Reasoning + Acting)

**What it is:**

- Agent alternates between reasoning and taking actions
- Each step: Think → Act → Observe → Repeat

**When to use:**

- Complex, multi-step tasks
- Need to adapt based on intermediate results
- Debugging unknown issues
- Exploratory coding

**How it works:**

```
Thought: I need to understand the bug first
Action: Read the error logs
Observation: NullPointerException in UserService.java line 45

Thought: The issue is a null user object, I should check where it's created
Action: Search for UserService instantiation
Observation: Found 3 places where User is created

Thought: Line 23 doesn't validate input, that's likely the bug
Action: Add null check at line 23
Observation: Code updated successfully

Thought: Should verify the fix works
Action: Run unit tests
Observation: All tests passing
```

**Pros:**

- Transparent reasoning
- Adapts to new information
- Good for complex problems

**Cons:**

- Slower (many steps)
- Can get stuck in loops

- More LLM calls = higher cost

**Example Implementation:**

```
class ReActAgent:
    def solve(self, task):
        state = {"task": task, "completed": False}

        while not state["completed"]:
            # Reasoning step
            thought = self.think(state)
            print(f"Thought: {thought}")

            # Action step
            action = self.decide_action(thought)
            print(f"Action: {action}")

            # Execute action
            observation = self.execute(action)
            print(f"Observation: {observation}")

            # Update state
            state = self.update_state(observation)

        return state["result"]
```

## 2. Plan-and-Execute

**What it is:**

- Create complete plan upfront
- Execute steps sequentially
- Re-plan if step fails

**When to use:**

- Well-defined tasks
- Known requirements
- Implementation with clear specs
- Building from scratch

**How it works:**

```
Plan:
1. Create User model class
2. Add database migration
3. Create UserService with CRUD operations
4. Add input validation
5. Write unit tests
6. Write integration tests
```

```
Execute:
[1/6] Creating User model... ✓
[2/6] Adding migration... ✓
[3/6] Creating UserService... ✓
[4/6] Adding validation... ✓
[5/6] Writing unit tests... ✓
[6/6] Writing integration tests... ✓
```

**Pros:**

- Clear progress tracking
- Predictable execution
- Efficient (fewer LLM calls)
- Good for teams (shows roadmap)

**Cons:**

- Rigid (hard to adapt mid-execution)
- Plan may be wrong
- Doesn't handle surprises well

**Example Implementation:**

```python
class PlanExecuteAgent:
    def solve(self, task):
        # Planning phase
        plan = self.create_plan(task)
        print(f"Plan: {plan}")

        # Execution phase
        for step in plan:
            try:
                result = self.execute_step(step)
                print(f"✓ {step}: {result}")
            except Exception as e:
                print(f"✗ {step} failed: {e}")
                # Re-plan from this point
                new_plan = self.replan(plan, step, e)
                plan = new_plan

        return self.get_result()
```

## 3. ReWOO (Reasoning WithOut Observation)

**What it is:**

- Plan all observations needed upfront
- Execute all in parallel
- Reason on combined results

**When to use:**

- Need to gather info from multiple sources
- Want parallelization
- Cost optimization (fewer sequential calls)
- Data aggregation tasks

**How it works:**

```
# Planning phase (single LLM call)
Plan:
- Evidence 1: Read user requirements from requirements.md
- Evidence 2: Check existing User model structure
- Evidence 3: Get database schema for users table
- Evidence 4: Review similar service implementations

# Execution phase (parallel)
[Parallel execution of all 4 evidence gathering]

# Reasoning phase (single LLM call with all evidence)
Given Evidence 1-4, implement UserService with:
- CRUD operations matching existing pattern
- Validation based on requirements
- Schema compatibility
```

**Pros:**

- Fewer LLM calls (2-3 total)
- Parallel execution = fast
- Cost effective

**Cons:**

- Can't adapt plan mid-execution
- May gather unnecessary info
- Complex to implement

**Example Implementation:**

```
class ReWOOAgent:
    def solve(self, task):
        # Phase 1: Plan what evidence is needed
        evidence_plan = self.plan_evidence(task)
        print(f"Evidence needed: {evidence_plan}")

        # Phase 2: Gather all evidence in parallel
        import asyncio
        evidence = asyncio.run(self.gather_evidence_parallel(evidence_plan))

        # Phase 3: Reason with all evidence
```

```
        solution = self.reason_with_evidence(task, evidence)
        return solution
```

### 4. Reflexion (Self-Critique)

**What it is:**

- Execute task
- Critique own work
- Refine and improve
- Repeat until satisfied

**When to use:**

- Code quality matters
- Need optimization
- Complex algorithms
- Production code

**How it works:**

```
Iteration 1:
Implementation: Basic UserService with CRUD
Critique:
  - Missing input validation
  - No error handling
  - Inefficient database queries
Refinement: Add validation and error handling

Iteration 2:
Implementation: UserService with validation and errors
Critique:
  - Still inefficient queries (N+1 problem)
  - Missing logging
  - No caching
Refinement: Optimize queries, add logging and caching

Iteration 3:
Implementation: Optimized UserService
Critique:
  - Code looks good
  - All best practices followed
  - Performance is optimal
Result: Accept and complete
```

**Pros:**

- High quality output
- Self-improving

- Catches mistakes

**Cons:**

- Multiple iterations = slow
- Expensive (many LLM calls)
- May over-engineer

**Example Implementation:**

```
class ReflexionAgent:
    def solve(self, task, max_iterations=3):
        solution = None

        for i in range(max_iterations):
            # Execute
            solution = self.implement(task, previous=solution)
            print(f"Iteration {i+1}: {solution}")

            # Self-critique
            critique = self.critique(solution)
            print(f"Critique: {critique}")

            # Check if satisfied
            if critique["quality"] >= 8:
                print("Quality threshold met")
                break

            # Refine for next iteration
            task = self.refine_task(task, critique)

        return solution
```

## 5. Tree of Thoughts (ToT)

**What it is:**

- Generate multiple solution approaches
- Explore each branch
- Evaluate and select best

**When to use:**

- Multiple valid approaches exist
- Need optimal solution
- Algorithm design
- Architecture decisions

**How it works:**

```
Task: Implement caching layer

Branch 1: Redis
  ├─ Distributed caching ✓
  ├─ Fast (in-memory) ✓
  ├─ Complex setup ✗
  └─ Score: 7/10

Branch 2: In-memory (dict)
  ├─ Simple ✓
  ├─ Fast ✓
  ├─ Not distributed ✗
  ├─ Memory limited ✗
  └─ Score: 5/10

Branch 3: Memcached
  ├─ Distributed ✓
  ├─ Simple protocol ✓
  ├─ No persistence ✗
  └─ Score: 6/10

Selected: Redis (highest score)
```

**Pros:**

- Explores alternatives
- Finds optimal solution
- Good for critical decisions

**Cons:**

- Very expensive (multiple branches)
- Slow (explores all options)
- Overkill for simple tasks

**Example Implementation:**

```python
class TreeOfThoughtsAgent:
    def solve(self, task):
        # Generate multiple approaches
        approaches = self.generate_approaches(task)
        print(f"Approaches: {approaches}")

        results = []
        for approach in approaches:
            # Explore this branch
            solution = self.explore_branch(approach)
            score = self.evaluate(solution)
            results.append((approach, solution, score))
            print(f"{approach}: score={score}")
```

```
        # Select best
        best = max(results, key=lambda x: x[2])
        return best[1]
```

## 6. Direct / Zero-Shot

**What it is:**

- Single prompt → single response
- No reasoning steps visible
- Direct implementation

**When to use:**

- Simple, well-defined tasks
- Need speed
- Cost-sensitive
- Boilerplate code

**How it works:**

```
Task: Add a new API endpoint for getting user by ID

Direct implementation:
@app.get("/users/{user_id}")
def get_user(user_id: int):
    user = db.query(User).filter(User.id == user_id).first()
    if not user:
        raise HTTPException(status_code=404)
    return user
```

**Pros:**

- Fastest mode
- Cheapest (single LLM call)
- Simple to implement

**Cons:**

- No reasoning shown
- Can't handle complexity
- No error recovery

**Example Implementation:**

```
class DirectAgent:
    def solve(self, task):
        # Single shot completion
```

```
        result = self.llm.complete(
            f"Implement the following: {task}"
        )
        return result
```

## Mode Selection Matrix

| Mode | Speed | Cost | Quality | Complexity | Best For |
|---|---|---|---|---|---|
| Direct | ★★★★★ | ★★★★★ | ★★ | ★ | Simple tasks |
| Plan-Execute | ★★★★ | ★★★★ | ★★★★ | ★★ | Feature development |
| ReAct | ★★ | ★★ | ★★★★ | ★★★★ | Debugging, exploration |
| ReWOO | ★★★★ | ★★★★★ | ★★★ | ★★★ | Data gathering |
| Reflexion | ★ | ★ | ★★★★★ | ★★★ | Production code |
| Tree of Thoughts | ★ | ★ | ★★★★★ | ★★★★★ | Architecture decisions |

## Real-World Examples

### Example 1: Bug Fix

**Scenario:** NullPointerException in production

**Best Mode:** ReAct

**Why:**

- Need to explore and understand
- Problem not clearly defined
- Must adapt based on findings

**Flow:**

```
Thought: Check error logs for stack trace
Action: Read logs/error.log
Observation: NPE at UserService.java:45

Thought: Check what's null at line 45
Action: Read UserService.java lines 40-50
Observation: user.getEmail() when user is null

Thought: Why is user null? Check caller
Action: Search for calls to this method
Observation: Called from AuthController without null check

Thought: Add validation before calling
Action: Add if (user == null) check in AuthController
Observation: Code updated
```

```
Thought: Verify fix
Action: Run tests
Observation: Tests pass
```

### Example 2: New Feature

**Scenario:** Implement user registration API

**Best Mode:** Plan-and-Execute

**Why:**

- Requirements are clear
- Standard implementation
- Can plan upfront

**Flow:**

```
Plan:
1. Create User model
2. Create registration DTO
3. Add password hashing
4. Implement registration endpoint
5. Add input validation
6. Write tests

Execute each step sequentially
```

### Example 3: Performance Optimization

**Scenario:** API endpoint is slow (5s response)

**Best Mode:** ReAct + Reflexion

**Why:**

- Need to investigate cause (ReAct)
- Then optimize iteratively (Reflexion)

**Flow:**

```
# ReAct phase
Thought: Add timing to identify bottleneck
Action: Add timing logs
Observation: Database query takes 4.5s

Thought: Check query
Action: Read query code
```

```
Observation: Missing index on user_id

# Reflexion phase
Iteration 1: Add index
Critique: Faster (2s) but still slow
Refinement: Cache frequent queries

Iteration 2: Add caching
Critique: Much faster (200ms) but high memory
Refinement: Add TTL and size limit

Iteration 3: Optimized caching
Critique: Fast (200ms), memory efficient ✓
```

### Example 4: Gather User Requirements

**Scenario:** Create technical spec from scattered docs

**Best Mode:** ReWOO

**Why:**

- Need info from multiple sources
- Can gather in parallel
- One final synthesis

**Flow:**

```
Evidence Plan:
1. Read requirements.md
2. Read architecture.md
3. Check existing API patterns
4. Review similar features
5. Get database schema

[Execute all 5 in parallel]

Synthesis:
Based on all evidence, create spec with:
- API design matching patterns (Evidence 3)
- Requirements coverage (Evidence 1)
- Architecture alignment (Evidence 2)
- Database compatibility (Evidence 5)
```

### Example 5: Choose Database

**Scenario:** Select database for new project

**Best Mode:** Tree of Thoughts

**Why:**

- Multiple valid options
- Important decision
- Need to compare thoroughly

**Flow:**

```
Explore PostgreSQL:
+ ACID compliance
+ Relational data
+ Complex queries
- Harder to scale
Score: 8/10

Explore MongoDB:
+ Flexible schema
+ Horizontal scaling
+ Fast reads
- Eventual consistency
- Complex transactions
Score: 6/10

Explore DynamoDB:
+ Fully managed
+ Auto-scaling
+ High performance
- Vendor lock-in
- Learning curve
Score: 7/10

Selected: PostgreSQL (best for our use case)
```

# Hybrid Approaches

## ReAct + Plan-Execute

**When:** Complex feature with unknowns

```
Phase 1 (ReAct): Explore codebase and requirements
Phase 2 (Plan-Execute): Implement based on findings
```

## Plan-Execute + Reflexion

**When:** Feature with high quality bar

```
Phase 1 (Plan-Execute): Implement feature
Phase 2 (Reflexion): Review and optimize
```

### ReWOO + ReAct

**When:** Data gathering followed by problem solving

```
Phase 1 (ReWOO): Gather all relevant info
Phase 2 (ReAct): Debug/implement with context
```

## Implementation Guidelines

### Choosing the Right Mode

**Ask yourself:**

1. **Is the task well-defined?**
    - Yes → Plan-Execute or Direct
    - No → ReAct or Reflexion
2. **Is quality critical?**
    - Yes → Reflexion or Tree of Thoughts
    - No → Direct or Plan-Execute
3. **Is it exploratory?**
    - Yes → ReAct
    - No → Plan-Execute
4. **Multiple sources needed?**
    - Yes → ReWOO
    - No → Other modes
5. **Multiple valid approaches?**
    - Yes → Tree of Thoughts
    - No → Other modes
6. **Budget/time constraints?**
    - Tight → Direct or ReWOO
    - Flexible → ReAct or Reflexion

### Cost Comparison

**Example: "Implement user registration"**

| Mode | LLM Calls | Approx Cost | Time |
|---|---|---|---|

| Mode | LLM Calls | Approx Cost | Time |
|---|---|---|---|
| Direct | 1 | $0.01 | 5s |
| Plan-Execute | 5-7 | $0.05 | 30s |
| ReAct | 8-12 | $0.10 | 60s |
| ReWOO | 3 | $0.03 | 20s |
| Reflexion | 12-20 | $0.15 | 90s |
| Tree of Thoughts | 15-30 | $0.25 | 120s |

## Interview Tips

**"What agent mode would you use for X?"**

**Framework:**

1. Define the task characteristics
2. Map to mode selection criteria
3. Justify choice
4. Mention alternatives and why not

**Example Answer:**

"For debugging a production issue, I'd use ReAct mode because:

- The problem isn't well-defined yet
- We need to adapt based on what we find
- It provides transparent reasoning
- Can handle unexpected findings

I wouldn't use Plan-Execute because we can't plan without understanding the issue first. Direct mode is too simplistic for debugging."

## Tools and Frameworks

**LangChain:**

```
from langchain.agents import AgentType, initialize_agent
from langchain.agents import load_tools

# ReAct agent
agent = initialize_agent(
    tools,
    llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    verbose=True
)

# Plan-and-execute agent
```

```
agent = initialize_agent(
    tools,
    llm,
    agent=AgentType.PLAN_AND_EXECUTE,
    verbose=True
)
```

**AutoGPT:**

- Autonomous ReAct-style agent
- Continuous operation
- Self-directed

**BabyAGI:**

- Task-driven autonomous agent
- Creates and prioritizes tasks
- Executes tasks iteratively

## Best Practices

1. **Start simple**: Use Direct/Plan-Execute for straightforward tasks
2. **Escalate complexity**: Move to ReAct/Reflexion only when needed
3. **Monitor costs**: Track LLM calls per mode
4. **Log reasoning**: Capture thought process for debugging
5. **Set limits**: Max iterations to prevent infinite loops
6. **Combine modes**: Use hybrid approaches for complex projects
7. **Measure quality**: Track output quality by mode
8. **User feedback**: Let users choose mode for their tasks